Design Decisions and Data Architecture

Design is a series of justified decisions,
not just implementation.

DAEN 459 – Fall 2026

Alexander Abuabara



1. Engineering principles

Engineering principles: What makes a solution sound?

- **Scientific reasoning**
  Use models, data, and evidence rather than assumptions.
- **Safety and responsibility**
  Protect users, data, and the public.
- **Reliability and verification**
  Measure whether the system performs as required.

2. Engineering design principles

Engineering design principles: How do we choose a solution?

- **Requirements first**
  Define stakeholders, needs, constraints, and success criteria.
- **Compare alternatives**
  Justify trade-offs in cost, quality, complexity, and risk.
- **Iterate and simplify**
  Prototype, test, refine; design for maintenance and change.

Engineering principles establish *what must hold*;
Design principles guide *how to decide what to build*.
A strong project explains both the choices and the evidence.

3. Engineering Design

1. What makes an activity *engineering design*, rather than only coding or analysis?
2. What problem are we solving, and for whom?
3. How do requirements, constraints, and trade-offs influence a design?
4. How can we know whether a design is good enough?

3. Engineering design

- **Meaning:** A systematic, iterative process of defining needs, creating alternatives, evaluating trade-offs, and validating a solution.
- **Starting point:** Stakeholders, problem statement, intended use, and measurable success criteria.
- **Constraints:** Time, cost, data access, reliability, privacy, skills, and maintainability.
- **Evidence:** Tests, benchmarks, reviews, prototypes, and feedback against requirements.

A design is not simply "the system we built." It includes *why* this solution was selected over plausible alternatives.

4. Design planning

1. What needs to be decided before implementation begins?
2. Which decisions must be made early, and which can wait?
3. What deliverables demonstrate design progress?
4. How should a team respond when assumptions turn out to be wrong?
5. Which unresolved question poses the largest risk to your project?

4. Design planning

- **Plan around decisions:** Scope, users, requirements, dependencies, responsibilities, and checkpoints.
- **De-risk early:** Investigate uncertain data sources, integrations, performance, and feasibility first.
- **Useful artifacts:** Requirement list, architecture sketch, data inventory, prototype, test plan, and decision log.
- **Iterate:** Revise designs based on evidence; document changes and their consequences.

A schedule says *when* work happens. A design plan also says *what must be learned and decided* before proceeding.

5. Engineering data architecture

1. What is data architecture, and how is it different from a database schema?
2. Where do data originate, move, change, and get consumed?
3. What components and interfaces does the system need?
4. How do reliability, security, and scale affect architectural choices?

This week: Let us draw the path of one record from its source to a user-facing result.

5. Engineering data architecture

- **Meaning:** The high-level organization of data sources, ingestion, storage, transformation, access, and governance.
- **Data flow:** Identify producers, consumers, movement, transformations, and boundaries between systems.
- **Choices:** Batch vs. streaming; files vs. databases; centralized vs. distributed processing; managed vs. self-hosted services.
- **Quality attributes:** Availability, latency, scalability, traceability, security, cost, and operability.

Architecture explains how the *parts work together*;
a schema specifies how data are *structured within those parts*.

6. Data schema

1. What does a schema specify about data?
2. Which entities, attributes, keys, and relationships matter?
3. How are missing values, data types, units, and constraints represented?
4. What happens when the data or requirements evolve?

Discuss: What error becomes possible if a key, unit, or constraint is missing?

6. Data schema

- **Meaning:** A formal description of data structure and rules, e.g., tables, fields, types, relationships, and constraints.
- **Model at several levels:** Conceptual entities; logical relationships; physical tables, columns, and indexes.
- **Specify meaning:** Names, definitions, units, allowed values, nullability, uniqueness, and referential integrity.
- **Plan for change:** Versioning, compatibility, migrations, and validation of incoming data.

A schema is an explicit agreement about what data *mean* and what the system is allowed to store.

7. Putting it together — Questions

1. What is the engineering problem and how will success be measured?
2. What should the design plan investigate first?
3. What data architecture would connect to the dashboard?
4. What schema would represent reliably?
5. Which design trade-off would you defend to a reviewer?

7. Putting it together — Possible answers

- **Engineering design:** Help facilities staff identify unusual consumption; measure freshness, coverage, and accuracy.
- **Design plan:** Check sensor access and sampling quality; build an end-to-end prototype before adding features.
- **Architecture:** Sensors → ingestion → validation → storage → analytics/API → dashboard.
- **Schema:** `Reading(sensor_id, observed_at, value, unit, quality_flag)`; define keys, types, and timestamp conventions.
- **Trade-off:** A 15-minute batch update may be cheaper and easier to maintain than real-time streaming, if users accept the delay.

There is no single perfect answer. Ask whether the team's assumptions and evidence justify each choice.

Complementary ideas

- **Engineering design**
  What solution should we build, and why?
- **Design planning**
  What must we decide, test, and deliver, and in what order?
- **Data architecture**
  How will data flow through the system?
- **Data schema**
  How is data represented, constrained, and interpreted?

What is one important design choice your team has made,
and what evidence supports it?